1. Find the Second Largest Element

Description

Given an array of integers, find the second largest distinct element in the array.

Input Format

First line contains an integer N.

Second line contains N space-separated integers.

Output Format

Print the second largest distinct element.

Sample Input

6

12 35 1 10 34 35

Sample Output

34

Explanation

The largest element is 35 and the second largest distinct element is 34.

PROGRAM:

```
#include <stdio.h>
int main()
{
   int n;
   scanf("%d",&n);
   int arr[n];
   for(int i=0;i<n;i++){
      scanf("%d",&arr[i]);
   }
   int fm;
   int sm;
```

```
    if(arr[0]>arr[1]){
        fm=arr[0];
        sm=arr[1];
    }else{
        fm=arr[1];
        sm=arr[0];
    }
    for(int i=2;i<n;i++){
        if(arr[i]>=fm){
            fm=arr[i];
        }else if(arr[i]>sm){
            sm=arr[i];
        }
    }

    printf("second max is %d",sm);

    return 0;
}
```

2. Move All Zeros to the End

Description

Move all zero elements to the end while maintaining the order of non-zero elements.

Input Format

N

N space-separated integers

Output Format

Print the modified array.

Sample Input

7

0 1 0 3 12 0 5

Sample Output

1 3 12 5 0 0 0

Explanation

All non-zero elements retain their order, and zeros are moved to the end.

PROGRAM:

```
#include <stdio.h>
int main()
{
   int n;
   scanf("%d",&n);
   int arr[n];
   for(int i=0;i<n;i++){
      scanf("%d",&arr[i]);
   }
   int k=0;

   for(int i=0;i<n;i++){
      if(arr[i]!=0){
         arr[k]=arr[i];
         k++;
      }
   }
   while(k<n){
      arr[k]=0;
      k++;
   }
   for(int i=0;i<n;i++){
      printf("%d ",arr[i]);
   }
```

```
    return 0;
}
```

3. Check if Array is Sorted

Description

Determine whether the array is sorted in ascending order.

Input Format

N

N space-separated integers

Output Format

Print YES if sorted, otherwise NO.

Sample Input

5

2 4 6 8 10

Sample Output

YES

Explanation

Every element is greater than or equal to the previous element.

PROGRAM:

```
#include <stdio.h>

int main()
{
    int n;
    scanf("%d",&n);
    int arr[n];
    for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
```

```
    }

    int k=0;
    for(int i=0;i<n-1;i++){
        if(arr[i]>arr[i+1]){
            k++;
        }
    }
    if(k==0){
        printf("YES");
    }else{
        printf("NO");
    }

    return 0;
}
```

4. Find Missing Number

Description

An array contains numbers from 1 to N with one number missing. Find the missing number.

Input Format

N

N-1 space-separated integers

Sample Input

5

1 2 3 5

Sample Output

4

Explanation

Numbers from 1 to 5 should be present. Number 4 is missing.

PROGRAM:

```
#include <stdio.h>


int main()
{
   int n;
   scanf("%d",&n);
   int arr[n];
   for(int i=0;i<n-1;i++){
      scanf("%d",&arr[i]);
   }
   int ans=0;
   for(int i=0;i<n-2;i++){
      if(arr[i]+1 != arr[i+1]){
         ans=arr[i]+1;
      }
   }
   printf("Missing number is %d",ans);


   return 0;
}
```

5. Rotate Array by K Positions

Description

Rotate the array to the right by K positions.

Input Format

N

N space-separated integers

K

Sample Input

5

1 2 3 4 5

2

Sample Output

4 5 1 2 3

Explanation

After rotating twice to the right, the last two elements move to the front.

PROGRAM:

```
#include <stdio.h>


int main()
{
   int n;
   scanf("%d",&n);
   int arr[n];
   for(int i=0;i<n;i++){
      scanf("%d",&arr[i]);
   }
   int x,k=0;
   scanf("%d",&x);
   int copy[n];
   for(int i=0;i<n;i++){
      if(k+x < n){
      copy[k+x]=arr[i];
      k++;
      }else{
         copy[(k+x)-n]=arr[i];
         k++;
      }
   }
```

```
    for(int i=0;i<n;i++){
        printf("%d ",copy[i]);
    }

    return 0;
}
```

6. Remove Duplicates from a Sorted Array

Description

Remove duplicate elements from a sorted array.

Input Format

N

N space-separated integers

Sample Input

8

1 1 2 2 3 4 4 5

Sample Output

1 2 3 4 5

Explanation

Duplicate values are removed, leaving only unique elements.

PROGRAM:

```
#include <stdio.h>

int main()
{
    int n;
    scanf("%d",&n);
    int arr[n];
    for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
```

```
    }
    for(int i=0;i<n-1;i++){
        if(arr[i]==arr[i+1]){
            arr[i]=-1000;
        }
    }
    for(int i=0;i<n;i++){
        if(arr[i]!=-1000){
            printf("%d ",arr[i]);
        }
    }

    return 0;
}
```

7. Find Leaders in an Array

Description

An element is a leader if it is greater than all elements to its right.

Input Format

N

N space-separated integers

Sample Input

6

16 17 4 3 5 2

Sample Output

17 5 2

Explanation

17 is greater than all elements to its right.

5 is greater than 2.

2 is always a leader because it is the last element.

PROGRAM:

```
#include <stdio.h>


int main()
{
   int n;
   scanf("%d",&n);
   int arr[n];
   for(int i=0;i<n;i++){
      scanf("%d",&arr[i]);
   }
   int k=0;
   for(int i=0;i<n;i++){
      for(int j=i+1;j<n;j++){
         if(arr[i]<arr[j]){
            k++;
         }
      }if(k==0){
         printf("%d ",arr[i]);
      }k=0;
   }



   return 0;
}
```

8. Find Frequency of Each Element

Description

Print the frequency of each distinct element.

Input Format

N

N space-separated integers

Sample Input

7

1 2 2 3 1 4 2

Sample Output

1 -> 2

2 -> 3

3 -> 1

4 -> 1

Explanation

Count the occurrences of each element.

PROGRAM:

```
#include <stdio.h>

int main()
{
   int n;
   scanf("%d",&n);
   int arr[n];
   for(int i=0;i<n;i++){
      scanf("%d",&arr[i]);
   }
   int k=1;
   for(int i=0;i<n;i++){
      for(int j=i+1;j<n;j++){
         if(arr[i]==arr[j] && arr[i]!= -10){
            k++;
```

```
            arr[j]=-10;
          }
      }if(arr[i]!= -10){
      printf("%d -> %d\n",arr[i],k);
      }
      k=1;
   }


   return 0;
}
```

9. Find Pair with Given Sum

Description

Check whether any pair of elements adds up to a given target.

Input Format

N

N space-separated integers

Target

Sample Input

5

2 7 11 15 3

9

Sample Output

YES

Explanation

2 + 7 = 9.

PROGRAM:

```
#include <stdio.h>
```

```
int main()
{
   int n;
   scanf("%d",&n);
   int arr[n];
   for(int i=0;i<n;i++){
      scanf("%d",&arr[i]);
   }
   int target;
   scanf("%d",&target);
   int k=0;
   for(int i=0;i<n;i++){
      for(int j=i+1;j<n;j++){
         if(arr[i]+arr[j]==target){
            k++;
         }
      }
   }if(k!=0){
      printf("YES");
   }else{
      printf("NO");
   }



   return 0;
}
```

10. Maximum Difference

Description

Find the maximum difference between two elements where the larger element comes after the smaller one.

Sample Input

7

2 3 10 6 4 8 1

Sample Output

8

Explanation

Maximum difference is 10 - 2 = 8.

PROGRAM:

```
#include <stdio.h>


int main()
{
   int n;
   scanf("%d",&n);
   int arr[n];
   for(int i=0;i<n;i++){
      scanf("%d",&arr[i]);
   }
   int max=arr[0],k=0;
   for(int i=0;i<n;i++){
      if(max<arr[i]){
         max=arr[i];
         k=i;
      }
   }int min=arr[0];
   for(int i=0;i<k;i++){
      if(min>arr[i]){
         min=arr[i];
```

```
    }
  }
  printf("difference is %d",max-min);

  return 0;
}
```

11. Product of Array Except Self

Description

For every index, find the product of all elements except itself.

Sample Input

4

1 2 3 4

Sample Output

24 12 8 6

Explanation

For index 0: 2×3×4 = 24

For index 1: 1×3×4 = 12

and so on.

PROGRAM:

```
#include <stdio.h>

int main()
{
  int n;
  scanf("%d",&n);
  int arr[n];
  for(int i=0;i<n;i++){
```

```
    scanf("%d",&arr[i]);
  }
  int k=1;
  for(int i=0;i<n;i++){
    for(int j=0;j<n;j++){
      if(j!=i){
        k*=arr[j];
      }
    }printf("%d ",k);
    k=1;
  }

  return 0;
}
```

12. Longest Consecutive Sequence

Description

Find the length of the longest consecutive sequence.

Sample Input

6

100 4 200 1 3 2

Sample Output

4

Explanation

Sequence 1, 2, 3, 4 has length 4.

PROGRAM:

```
#include <stdio.h>

int main()
{
```

```
    int n;
    scanf("%d",&n);
    int arr[n];
    for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }
    int k=1;
    for(int i=0;i<n;i++){
        for(int j=i+1;j<n;j++){
            if(arr[i]>arr[j]){
                int temp=arr[i];
                arr[i]=arr[j];
                arr[j]=temp;
            }
        }
    }
    for(int i=0;i<n;i++){
        if(arr[i]==arr[i+1]-1){
            k++;
        }
    }printf("longest consecutive is %d",k);

    return 0;
}
```

13. Equilibrium Index

Description

Find an index where left sum equals right sum.

Sample Input

5

1 3 5 2 2

Sample Output

2

Explanation

Left sum = 1 + 3 = 4

Right sum = 2 + 2 = 4

PROGRAM:

```
#include <stdio.h>

int main()
{
   int n;
   scanf("%d",&n);
   int arr[n];
   for(int i=0;i<n;i++){
      scanf("%d",&arr[i]);
   }
   int mid=n/2,sum1=1,sum2=0;
   while(sum1!=sum2){
      sum1=0;
      sum2=0;
   for(int i=0;i<n;i++){
      if(i<mid){
         sum1+=arr[i];
      }else if(i>mid){
         sum2+=arr[i];
      }
   }if(sum1<sum2){
      mid++;
   }else if(sum1>sum2){
      mid--;
```

```
    }
    }printf("the index is %d",mid);


    return 0;
}
```

14. Rearrange Positive and Negative Numbers

Description

Arrange positive and negative numbers alternately.

Sample Input

6

1 -2 3 -4 5 -6

Sample Output

1 -2 3 -4 5 -6

Explanation

The array is already in alternating order.

PROGRAM:

```
#include <stdio.h>


int main()
{
    int n;
    scanf("%d",&n);
    int arr[n];
    for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }
    int copy[n];
    int k=0,l=1;
    for(int i=0;i<n;i++){
```

```
        if(arr[i]>0){
            copy[k]=arr[i];
            k+=2;
        }
        else if(arr[i]<0){
            copy[l]=arr[i];
            l+=2;
        }
    }
    for(int i=0;i<n;i++){
        printf("%d ",copy[i]);
    }

    return 0;
}
```

15. Majority Element

Description

Find the element appearing more than N/2 times.

Sample Input

7

2 2 1 2 3 2 2

Sample Output

2

Explanation

2 appears 5 times, which is more than 7/2.

PROGRAM:

```
#include <stdio.h>
```

```
int main()
{
   int n;
   scanf("%d",&n);
   int arr[n];
   for(int i=0;i<n;i++){
      scanf("%d",&arr[i]);
   }
   int c=1;
   for(int i=0;i<n;i++){
      for(int j=i+1;j<n;j++){
         if(arr[i]==arr[j] && arr[i]!= -10){
            c++;
            arr[j]=-10;
         }
      }if(c>n/2){
         printf("Majority element is %d",arr[i]);
         return 0;
      }
      c=1;
   }

   return 0;
}
```

16. Maximum Sum Subarray

Description

Find the contiguous subarray with the maximum sum.

Sample Input

9

-2 1 -3 4 -1 2 1 -5 4

Sample Output

6

Explanation

Subarray [4, -1, 2, 1] has maximum sum 6.

PROGRAM:

```
#include <stdio.h>


int main()
{
   int n;
   scanf("%d",&n);
   int arr[n];
   for(int i=0;i<n;i++){
      scanf("%d",&arr[i]);
   }
   int max=arr[0],maxsum=arr[0];
   for(int i=1;i<n;i++){
      int x=max+arr[i];
      if(x>arr[i]){
         max+=arr[i];
      }else{
         max=arr[i];
      }
      if(max>maxsum){
         maxsum=max;
      }
   }
   printf("Maximum sum subarray %d ", maxsum);


   return 0;
```

}

17. Container With Most Water

Description

Find two lines that can contain the maximum water.

Sample Input

9

1 8 6 2 5 4 8 3 7

Sample Output

49

Explanation

Lines at indices 1 and 8 hold:

min(8,7) × (8-1) = 49

PROGRAM:

```
int maxArea(int* arr, int n) {
   int max=0,ans=0,i=0,j=n-1;
   while(i<j){
        if(arr[i]<=arr[j]){
           max=arr[i]*(j-i);
        }else if(arr[i]>arr[j]){
           max=arr[j]*(j-i);
        }
        if(max>ans){
           ans=max;
        }
        if(arr[i]<arr[j]){
           i++;
        }else{
           j--;
        }
```

```
    }
    return ans;

}
```

18. Trapping Rain Water

Description

Calculate total water trapped between bars.

Sample Input

12

0 1 0 2 1 0 1 3 2 1 2 1

Sample Output

6

Explanation

Total trapped water equals 6 units.

19. Merge Overlapping Intervals

Description

Merge all overlapping intervals.

Sample Input

4

1 3

2 6

8 10

15 18

Sample Output

1 6

8 10

15 18

Explanation

Intervals [1,3] and [2,6] overlap and are merged.

20. Maximum Product Subarray

Description

Find the contiguous subarray having the maximum product.

Sample Input

4

2 3 -2 4

Sample Output

6

Explanation

Subarray [2,3] gives the maximum product 6.